TECRES

...go beyond paper

TECHNICAL DOCUMENTATION

TecresTrackIT AMS Engineering Reference

Architecture, data model, authentication, multi-tenancy, the permission system, the asset and audit workflows, the HTTP API, and how to run, test and deploy both applications.

INTERNAL: TECRES ENGINEERING

TecresTrackIT AMS
Version 1.0 · 26 September 2026

...go beyond paper
Tecres

About this document

This is the engineering reference for TecresTrackIT AMS. It is written for developers joining the project and for anyone reviewing how the system works. It describes the code as of backend commit `cef8df1` (plus the token-audience security fix of 26 September 2026) and frontend commit `1c21661`. Where this document and the code disagree, the code wins; the repositories' `CLAUDE.md` and `BUILD_PLAN.md` files track day-to-day status.

Contents

1 Overview and stack

2 Repository layout

3 Request lifecycle

4 Data model

5 Authentication and sessions

6 Multi-tenancy and authorization

7 Domain workflows

8 Frontend architecture

9 HTTP API reference

10 Email, files and background jobs

11 Local setup and configuration

12 Testing and quality gates

13 Conventions for contributors

ENGINEERING

1 Overview and stack

TecresTrackIT AMS is a multi-tenant IT asset management platform. Customer organizations (tenants) track their assets through check-out, maintenance, audits, reporting and labelling. Tecres staff run the service from a separate platform console.

The system is two applications in two repositories that talk over a JSON API:

LAYER	REPOSITORY	TECHNOLOGY
Frontend SPA	`asset-management-frontend`	Vite, React 19, TypeScript, React Router 7, TanStack Query (server state), Zustand (client state), Tailwind CSS v4, shadcn/ui (Nova preset, Radix primitives), react-hook-form + zod, recharts, jspdf, xlsx, jsbarcode, qrcode, html5-qrcode. Package manager: **npm**.
Backend API	`asset-management-backend`	Node 20+, Express 4, TypeScript, Drizzle ORM on PostgreSQL, Zod, argon2, jsonwebtoken, otplib, Helmet, express-rate-limit, pino, nodemailer / Brevo, Cloudinary, AWS S3. Package manager: **pnpm**.
Database	n/a	PostgreSQL, one database shared by all tenants; every tenant table carries `org_id`.



Design principles

- **The server is the only security boundary.** The frontend hides what a role cannot use, but every rule is enforced again in the API.
- **Tenant isolation is structural.** The organization comes from the verified token, never from the request, and queries go through an org-scoped helper.
- **Two identities, never mixed.** Tenant users and Tecres platform admins have separate tables, secrets, middleware, stores and API clients.
- **Mandatory two-factor authentication** for every account, before any session exists.
- **Workspaces are provisioned by Tecres.** There is no public sign-up; a platform admin creates an organization and its first super admin.

ENGINEERING

2 Repository layout

Backend

```
src/
  config/        env.ts (Zod-validated environment), database.ts (Drizzle + pg Pool)
  db/
    schema/      one file per domain: organizations, users, assets, audits, security, platform, ...
    migrations/  drizzle-kit generated SQL, applied by migrate.ts
    seed.ts, seedPlatformAdmin.ts
  middleware/    authenticate, authenticatePlatformAdmin, loadOrgContext, authorize,
                 rateLimit, validate, errorHandler
  lib/           jwt, password, totp, crypto, permissions, scopedDb, crudFactory,
                 storage, cloudinary, email, activityLog, assetReportQuery, ...
  modules/       one folder per domain: <name>.router.ts + .service.ts + .schema.ts
  jobs/          alertDetection.cron.ts, tokenCleanup.cron.ts
  emails/        HTML templates with {{var}} placeholders, rendered through layout.html
  routes/        index.ts mounts every module router under /api
  test/          vitest setup + security/pentest.security.test.ts
  app.ts         Express app and middleware chain
  server.ts      entrypoint: HTTP server + background jobs
```

Frontend

```
src/
  app/router.tsx        route tree: tenant routes (ProtectedRoute / PermissionRoute + AppShell),
                        /platform/* behind PlatformProtectedRoute + PlatformShell
  features/<domain>/     pages + api.ts (TanStack Query hooks) + components/
  components/ui/        shadcn primitives (CLI-managed, do not hand-edit)
  components/layout/    AppShell, Sidebar, Header, MobileNav, CommandPalette, PlatformShell
  components/common/    StatusBadge family, PageHeader, ...
  components/brand/     Logo, BrandedLoader, OrgSwitcher
  lib/                  api-client.ts, platform-api-client.ts, permissions.ts (UX only),
                        format.ts, status.ts, depreciation.ts, env.ts
  store/                auth.store.ts, platformAuth.store.ts, theme.store.ts
  types/                domain types, kept in sync by hand with the backend schema
```

ENGINEERING

3 Request lifecycle

Every tenant request passes through the same ordered chain. Knowing the order explains most behavior, for example why a suspended organization is refused before any permission check.

#	STAGE	WHAT IT DOES
1	`helmet()`	Security headers (CSP, HSTS, frame and sniffing protection)
2	`cors()`	Allows only `CORS_ALLOWED_ORIGINS`, with credentials
3	`pinoHttp`	Request ID and structured logging (4xx at warn, 5xx at error)
4	`globalRateLimiter`	600 requests per 15 minutes per IP
5	`express.json`, `cookieParser`, `compression`	2 MB JSON limit; reads the refresh cookie
6	`authenticate`	Verifies the access JWT (secret + audience) and sets `req.user = { id, orgId, role }`
7	`loadOrgContext`	Loads the organization from the token's `orgId`; rejects suspended orgs with 403 `ORG_SUSPENDED`
8	`authorize(permission)`	Checks the role's permission set; 403 if missing
9	`validate({ body, query, params })`	Parses input with Zod and replaces it with the parsed (stripped) value
10	Service function	Business logic and database access
11	`errorHandler`	Maps Zod errors to 422 and `AppError`s to their status; anything else becomes a generic 500

A TYPICAL ROUTE · modules/assets/assets.router.ts

```
assetsRouter.use(authenticate, loadOrgContext);

assetsRouter.post("/", authorize("canCreateAsset"), validate({ body: assetCreateSchema }), async (req, res) => {
  const asset = await assetsService.createAsset(req.user!.orgId, req.body, req.user!.id);
  res.status(201).json({ data: stripValuationFields(asset, req.user!.role) });
});
```

Response and error envelope

SUCCESS

```
{ "data": { ... } }
// lists may add
{ "data": [...], "pagination": { ... } }
```

ERROR

```
{ "error": {
    "code": "VALIDATION_ERROR",
    "message": "Validation failed",
    "details": { ... } } }
```

STATUS	CODE	WHEN
401	`UNAUTHORIZED`	Missing, invalid or expired token; bad credentials
403	`FORBIDDEN`, `ORG_SUSPENDED`	Role lacks the permission; organization suspended
404	`NOT_FOUND`	No such record in *your* organization (another org's record is also 404, never 403)
409	`CONFLICT`	Unique constraint (duplicate email, slug, tag, code)
422	`VALIDATION_ERROR`	Input failed its Zod schema
429	n/a	Rate limit exceeded

ENGINEERING

4 Data model

The schema is defined with Drizzle in `src/db/schema/*.ts`; migrations are generated with `pnpm db:generate` and applied with `pnpm db:migrate` (also run automatically by `pnpm start`). IDs are application-generated text keys.

Tenant tables (all carry `org_id`)

TABLE	PURPOSE AND NOTABLE COLUMNS
`organizations`	The tenant. `slug` (unique), branding (`logo_url` , `primary_color` , `accent_color` , `sidebar_color`), currency, timezone, `asset_quota` / `user_quota` , `plan_status` (active / suspended / expired), `onboarding_completed_at` , label settings
`users`	Tenant staff. `email` **globally unique** (one email belongs to exactly one workspace, which is how login finds the org), `role` , `password_hash` , `totp_enabled` , `failed_login_attempts` , `locked_until` , `is_active`
`departments` , `locations` , `categories` , `vendors`	Catalog lookups (categories are shown as "asset types" in the UI)
`subsidiaries` , `branches`	Org structure used by structured asset tags; branches are org-level, codes unique per org
`asset_tag_sequences`	Running tag index per `(org_id, category_id)` , incremented atomically with `INSERT ... ON CONFLICT DO UPDATE`
`assets`	The asset register. `asset_tag` unique per org, `status` (available, in_use, in_maintenance, reserved, disposed, lost), `condition` , valuation fields, custodian, category/location/department/vendor
`asset_assignments`	Check-out / check-in records with due dates
`maintenance_records`	Status (scheduled, in_progress, waiting_parts, completed, cancelled), priority, cost
`audits` , `audit_items` , `audit_status_history`	Audit header, per-asset results (pending, matched, missing, relocated, damaged), and an append-only status log
`stored_files` , `asset_files`	Uploaded files (Cloudinary, S3 or `bytea`) and their link to assets
`activity_logs` , `notifications` , `system_settings`	Audit trail (including sign-in events with IP), user notifications, settings
`org_invites`	Staff invitations: SHA-256 token hash, role, expiry, resend count

Security tables

TABLE	PURPOSE
`refresh_tokens`	SHA-256 hash, `family_id` (rotation chain), `replaced_by_token_id` , `revoked_at` , IP, user agent
`totp_credentials`	AES-256-GCM encrypted secret, status (pending / active), `last_used_step` for replay protection
`totp_recovery_codes`	SHA-256 hash per code, `used_at`
`password_reset_tokens`	SHA-256 hash, 60-minute expiry, `used_at` , requesting IP

Platform tables (no `org_id`, separate identity space)

TABLE	PURPOSE
`platform_admins`	Tecres staff: role (platform_owner / platform_support), password hash, encrypted TOTP secret, hashed recovery codes
`platform_admin_invites`	Invitations for new platform admins (never reuses `org_invites`)
`platform_admin_activity_logs`	Every platform action with actor and target org/user
`platform_alerts`	Raised by the alert-detection job: quota_breach, plan_expired, stale_org, failed_login_spike

ENGINEERING

5 Authentication and sessions

5.1 Sign-in sequence

#	CALL	RESULT
1	`POST /api/auth/login` { email, password }	Finds the user by email (which identifies the org), checks lockout, verifies argon2id. First sign-in: `{ status: "totp_enrollment_required", enrollmentToken }`. Otherwise `{ status: "totp_challenge_required", preAuthToken }`. All failures: 401 "Invalid email or password".
2a	`POST /api/auth/totp/enroll` { enrollmentToken }	Creates a pending encrypted secret; returns QR code data URL and manual key
2b	`POST /api/auth/totp/verify-enroll` { enrollmentToken, code }	Activates TOTP, issues 10 recovery codes **and** a session
3	`POST /api/auth/login/totp` { preAuthToken, code }	Verifies the code (or a recovery code), stamps `last_used_step`, issues a session
4	`POST /api/auth/refresh` (cookie)	Rotates the refresh token; returns a new access token

A **session** is a 15-minute access JWT (returned in the body, kept in memory by the SPA) plus a 30-day opaque refresh token in an `httpOnly` cookie scoped to `/api/auth`.

5.2 Token types

TOKEN	SECRET	AUDIENCE	LIFETIME	CLAIMS
Tenant access	`JWT_ACCESS_SECRET`	`tecres:tenant:access`	15 min	sub, orgId, role
Tenant pre-auth	`JWT_ACCESS_SECRET`	`tecres:tenant:pre-auth`	5 min (10 enroll)	sub, purpose
Platform access	`PLATFORM_ADMIN_JWT_SECRET`	`tecres:platform:access`	15 min	sub, role
Platform pre-auth	`PLATFORM_ADMIN_JWT_SECRET`	`tecres:platform:pre-auth`	5 min (10 enroll)	sub, purpose
Refresh	n/a (random 256-bit)	n/a	30 days	opaque; stored as SHA-256

> **NEVER SHARE AN AUDIENCE**
>
> Each verifier accepts exactly one audience. A new token type must get its own audience constant in `src/lib/jwt.ts`; reusing one lets a token be accepted where it was never meant to be (see the Security Remediation Register, item F-01).

5.3 Refresh rotation and reuse detection

REFRESHSESSION · modules/auth/auth.service.ts (abridged)

```
const [existing] = await db.select().from(refreshTokens).where(eq(refreshTokens.tokenHash, sha256Hex(raw)));
if (!existing) throw new UnauthorizedError("Invalid refresh token");
if (existing.revokedAt) {                      // already rotated: someone replayed it
  await revokeWholeFamily(existing.familyId);   // forces full re-login
  throw new UnauthorizedError("Refresh token reuse detected");
}
// insert new token in the same family, mark the old one revoked + replacedBy
```

5.4 Password reset and forced reset

- `POST /auth/password-reset/request` always answers 200 with the same message (no account discovery), and emails a 60-minute single-use link to the user.
- `POST /auth/password-reset/confirm` sets the new password and revokes every refresh token.
- A platform owner can force a reset (`POST /platform/orgs/:id/users/:userId/force-reset`): sessions are revoked immediately and the user is emailed a reset link.

5.5 Platform admin authentication

Same two-step shape under `/api/platform/auth/*`, but platform sessions are a single 15-minute JWT with **no refresh endpoint**: a reload or expiry means signing in again. New platform admins join by invitation (`POST /platform/admins/invite` → emailed link → `POST /platform/admins/invite/:token/accept` sets a password, then TOTP enrollment).

ENGINEERING

6 Multi-tenancy and authorization

6.1 Org scoping

All tenants share one database. Three rules keep them apart; break none of them:

1. `orgId` comes **only** from `req.user.orgId` (the verified token). No tenant route accepts it from the body, query or params.
2. Tenant data access goes through `scopedDb(orgId)` wherever the resource fits its shape, so the org filter cannot be forgotten.
3. A lookup by ID in another org returns 404, not 403, so IDs of other tenants' records are not confirmed to exist.

USING SCOPEDDB · any tenant service

```
const scoped = scopedDb(orgId);
const location = await scoped.findById(locations, id);        // WHERE org_id = $1 AND id = $2
if (!location) throw new NotFoundError();
await scoped.update(locations, id, { name, updatedAt: new Date() });
await scoped.insert(departments, { id: generateId(), name });  // orgId injected automatically
```

6.2 Simple CRUD resources

Catalog-style resources use `createCrudRouter()`, which wires list/get/create/update/delete through `scopedDb`, maps unique violations to 409, and lets any org member read while requiring a permission to write:

MODULES/CATALOG/CATALOG.ROUTER.TS

```
export const locationsRouter = createCrudRouter({
  table: locations,
  createSchema: locationCreateSchema,
  updateSchema: locationUpdateSchema,
  writePermission: "canManageSettings",
});
```

6.3 Roles and permissions

The matrix lives in `backend/src/lib/permissions.ts`. `super_admin` and `admin` share one permission set; the difference is organizational (the first admin created with the workspace is a super admin).

PERMISSION KEY	SUPER ADMIN / ADMIN	ASSET MANAGER	STAFF
`canManageOrg`, `canManageSettings`	✓		
`canCreateAsset`, `canEditAsset`	✓	✓	
`canDeleteAsset`, `canEditAssetIdentity`	✓		
`canCheckInOut`	✓	✓	
`canScheduleMaintenance`	✓	✓	✓
`canCompleteMaintenance`	✓	✓	
`canManagePeople`	✓		
`canManageVendors`, `canExportReports`, `canPrintLabels`	✓	✓	
`canAssignAudits`, `canVerifyAudits`	✓	✓	
`canPerformAssignedAudit`	✓	✓	✓
`canViewAssetValuation`	✓	✓	

Valuation hiding. For roles without `canViewAssetValuation`, `stripValuationFields()` deletes `purchasePrice`, `currentValue`, `salvageValue` and `totalValuation` from responses. Any new money field must be added to `VALUATION_FIELDS` and routed through that function.

Frontend copy. `frontend/src/lib/permissions.ts` mirrors the matrix to hide navigation and buttons. It is UX only; when the backend matrix changes, update the copy, but never rely on it for protection.

6.4 Platform permissions

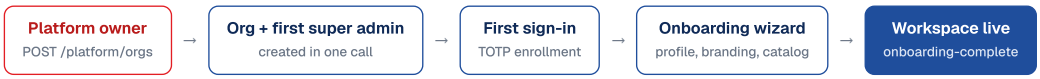
PERMISSION	PLATFORM_OWNER	PLATFORM_SUPPORT
View dashboard, organizations, users, activity, login risk, alerts	✓	✓

`canResolveAlert`	✓	✓
`canSuspendOrg` (also gates creating organizations)	✓	
`canAdjustQuota`, `canForceResetPassword`, `canManagePlatformAdmins`	✓	

ENGINEERING

7 Domain workflows

7.1 Workspace provisioning



The platform admin sets the first admin's credentials and hands them over out of band. Until `onboarding_completed_at` is set, the SPA routes that admin to `/onboarding`. Further staff join through `POST /people/invite` (emailed link, user sets their own password) or are added directly by an admin with `POST /people`.

7.2 Structured asset tags

When an organization has at least one subsidiary configured, new asset tags are generated server-side:

```
{SUB}-{TYPE}-{BRANCH}-{DDMMYY}-{index}[-{suffix}]        e.g. CCL-FF-09-210824-004
// index : running counter per asset type across the whole org (asset_tag_sequences)
// suffix: A, B, C... only when staff mark the asset as a duplicate of an existing one
```

Once set, tag-driving fields are locked on `PUT /assets/:id`. Corrections go through `PATCH /assets/:id/identity` (`canEditAssetIdentity`), which regenerates the tag and writes an activity-log entry. `GET /asset-tags/preview` returns an advisory preview for the form. Organizations without subsidiaries keep a free-text tag.

7.3 Check-out and return

`POST /assignments/checkout` (`assetId`, `userId`, optional `expectedReturnDate`, `conditionOut`, `notes`) sets the asset to *in_use*; `POST /assignments/checkin` (`assignmentId`, `conditionIn`, optional `notes`) returns it to *available*. `GET /assignments/:id/receipt` supplies data for the PDF receipt the frontend renders with jspdf.

7.4 Maintenance

Moving a record to *in_progress* sets the asset to *in_maintenance*; *completed* or *cancelled* returns it to *available*. Staff may schedule maintenance; updating or completing a record needs `canCompleteMaintenance`.

7.5 Audit state machine



TRANSITION	ROUTE	WHO
create (draft)	`POST /audits`	`canAssignAudits`
assign	`PATCH /audits/:id/assign`	`canAssignAudits`
start, scan items, submit, resume	`POST /audits/:id/start | scan | submit | resume`	The assigned auditor (`canPerformAssignedAudit`)
verify / reject	`POST /audits/:id/verify | reject`	`canVerifyAudits`, and **never the submitter**
cancel	`POST /audits/:id/cancel`	`canAssignAudits`

Every transition is appended to `audit_status_history`. The segregation-of-duties check (verifier ≠ submitter) is enforced in `audits.service.ts` regardless of role.

ENGINEERING

8 Frontend architecture

8.1 Routing and guards

GUARD	BEHAVIOR
`ProtectedRoute`	Waits for the boot-time silent refresh; signed out → `/login`; onboarding incomplete → `/onboarding`; otherwise renders inside `AppShell`
`PermissionRoute`	As above, plus redirects home if the role lacks the named permission (UX only)
`OnboardingRoute`	Session required, no onboarding redirect loop
`PlatformProtectedRoute`	Synchronous check of the platform store; renders inside the minimal `PlatformShell`

8.2 Sessions in the browser

- **Tokens live in memory** (Zustand), never `localStorage`. On boot, `auth.store.bootstrap()` calls `/auth/refresh` using the httpOnly cookie to restore the session.
- `apiFetch` attaches the bearer token and, on a 401, performs one refresh and retries. Concurrent refreshes are de-duplicated into a single in-flight promise, because the backend treats a second use of a rotated token as theft.
- The platform console uses `platformApiFetch` and `platformAuth.store.ts`. A 401 there simply signs the admin out (no refresh endpoint). **Never** share a store, token or client between the two identities.

TENANT API CLIENT · src/lib/api-client.ts

```ts
export async function apiFetch<T>(path: string, init: RequestInit = {}): Promise<T> {
  let res = await doFetch(path, init);
  if (res.status === 401) {
    const restored = await useAuthStore.getState().refresh();   // de-duplicated
    if (restored) res = await doFetch(path, init);
    else { useAuthStore.getState().clearSession(); throw new ApiError(401, "UNAUTHORIZED", "Your session has expired."); }
  }
  if (!res.ok) throw await ApiError.fromResponse(res);
  return res.status === 204 ? (undefined as T) : res.json();
}
```

8.3 Server state with TanStack Query

Each feature owns an `api.ts` of query and mutation hooks. Mutations invalidate every query key they can affect.

SRC/FEATURES/MAINTENANCE/API.TS (abridged)

```ts
export function useMaintenanceQuery() {
  return useQuery({
    queryKey: ["maintenance"],
    queryFn: () => apiFetch<{ data: MaintenanceRecord[] }>("/maintenance").then((r) => r.data),
  });
}

export function useUpdateMaintenanceMutation() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: ({ id, input }) => apiFetch(`/maintenance/${id}`, { method: "PUT", body: JSON.stringify(input) }),
    // status changes also flip the asset's status, so refresh both
    onSuccess: () => { queryClient.invalidateQueries({ queryKey: ["maintenance"] });
                       queryClient.invalidateQueries({ queryKey: ["assets"] }); },
  });
}
```

8.4 Theming and per-organization branding

`src/index.css` defines `--org-primary` and `--org-accent` as the only two colors that vary per tenant; every other token (`--primary`, `--ring`, sidebar, charts) derives from them. `useThemeStore().applyOrgTheme(org)` is the single place that writes them to the document. Light and dark palettes are both fully defined: the theme follows the device until the user picks one in the header, which is saved to `localStorage.tecres_theme` (a preference, not a secret). The sign-in page looks up the workspace from the email as it is typed and shows that organization's logo and color before sign-in.

8.5 UI conventions

- shadcn/ui components are added with `npx shadcn@latest add <name>` into `src/components/ui`; class merging uses the `cn` package; Radix comes from the unified `radix-ui` package.
- No em dashes in any user-visible text; use `"-"` for empty values.
- Reports, receipts and labels are rendered client-side (jspdf, xlsx, jsbarcode, qrcode); the backend returns JSON only.

ENGINEERING

9 HTTP API reference

Base path `/api`. Tenant routes need `Authorization: Bearer <access token>` and are org-scoped automatically. The "Guard" column shows the permission beyond authentication; a dash means any signed-in member. A runnable Bruno collection covering every route is in `backend/docs/api/asset-mgt/`.

Authentication

ROUTE	GUARD	NOTES
`POST /auth/login`	public, rate-limited	{ email, password } → enrollment or pre-auth token
`POST /auth/login/totp`	pre-auth token	{ preAuthToken, code } → session; code may be a recovery code
`POST /auth/totp/enroll`	enrollment token	Returns QR data URL + manual key
`POST /auth/totp/verify-enroll`	enrollment token	Activates TOTP; returns session + recovery codes
`POST /auth/totp/disable`	session	{ password, code }
`POST /auth/refresh`	refresh cookie	Rotates refresh token
`POST /auth/logout`, `/logout-all`	cookie / session	Revoke this session / all sessions
`POST /auth/password-reset/request`, `/confirm`	public, rate-limited	Always 200 on request; single-use token on confirm
`GET /auth/me`	session	Current user (no password hash)

Organization and people

ROUTE	GUARD	NOTES
`GET /organizations/public/lookup?email=`	public, 60/15 min	Workspace branding for the sign-in page
`GET /organizations/me`	-	Current organization
`PATCH /organizations/me`, `/me/theme`, `/me/label-format`, `/me/label-sheet-layout`, `/me/onboarding-complete`; `POST /me/logo`	canManageOrg	Profile, colors, label settings, logo upload (image, 5 MB)
`GET /organizations/me/label-sheet-layout`	canPrintLabels	Print calibration
`GET /people`, `GET /people/:id`	-	Directory
`POST /people`, `PUT /people/:id`, `DELETE /people/:id`	canManagePeople	Add directly, edit role/status, deactivate
`POST /people/invite`; `GET /people/invites`; `POST /people/invites/:id/resend`; `DELETE /people/invites/:id`	canManagePeople	Invitation lifecycle
`POST /people/invite/:token/accept`	invite token	{ name, password }

Catalog and structure

ROUTE	GUARD (WRITES)	NOTES
`/departments`, `/locations`, `/categories`	canManageSettings	Standard CRUD (`GET`, `GET /:id`, `POST`, `PUT /:id`, `DELETE /:id`)
`/vendors`	canManageVendors	Standard CRUD
`/subsidiaries`, `/branches`	canManageSettings	CRUD; codes locked once used in a tag

`GET /asset-tags/preview`	-	Advisory tag preview

Assets and operations

ROUTE	GUARD	NOTES
`GET /assets`, `GET /assets/:id`, `/:id/history`, `/:id/maintenance`	-	Paginated list with filters; valuation stripped for staff
`POST /assets`	canCreateAsset	Structured tag generation when subsidiaries exist
`PUT /assets/:id`	canEditAsset	Tag-identity fields locked
`PATCH /assets/:id/identity`	canEditAssetIdentity	Correct a locked tag; audit-logged
`DELETE /assets/:id`	canDeleteAsset	
`GET /assignments`; `GET /assignments/:id/receipt`	-	
`POST /assignments/checkout`, `/checkin`	canCheckInOut	{ assetId, userId, expectedReturnDate? } / { assignmentId, conditionIn }
`GET /maintenance`; `POST /maintenance`	- / canScheduleMaintenance	Anyone may request maintenance
`PUT /maintenance/:id`, `DELETE /maintenance/:id`	canCompleteMaintenance	Status changes also update the asset
`/audits` (see 7.5)	varies	Plus `GET /audits/trail`, `/:id/history`, `/:id/report`
`POST /files/upload`; `GET /files/:id`	-	20 MB; download re-checks org ownership

Insight

ROUTE	GUARD	NOTES
`GET /dashboard/stats`, `/activity`, `/charts`	-	KPIs, recent activity, chart series
`GET /reports/assets`	canExportReports	Filters, `dateField`, `sortBy` / `sortDir`
`GET /reports/valuation-depreciation`	+ canViewAssetValuation	
`GET /labels/assets`	canPrintLabels	Same filters as reports
`GET /search?q=`; `GET /activity`; `/notifications`	-	Global search, activity log, notifications (mark read)
`GET /settings`	canManageSettings	

Platform console (`/platform`)

ROUTE	GUARD	NOTES
`POST /auth/login`, `/auth/login/totp`, `/auth/totp/enroll`, `/auth/totp/verify-enroll`	public / pre-auth	Platform sign-in
`GET /admins/invite/:token`; `POST /admins/invite/:token/accept`	invite token	Join as a platform admin
`GET /stats`, `/analytics`, `/orgs`, `/orgs/:id`, `/orgs/:id/users`, `/orgs/:id/activity`, `/security/login-risk`, `/alerts`	platform session	Read-only fleet views
`POST /orgs`; `PATCH /orgs/:id/status`	canSuspendOrg	Create org + first admin; suspend / reactivate
`PATCH /orgs/:id/quotas`	canAdjustQuota	Asset and user quotas

`POST /orgs/:id/users/:userId/force-reset`	canForceResetPassword	Revoke sessions + email reset link
`GET /admins` ; `POST /admins/invite` ; `PATCH /admins/:id/role`	canManagePlatformAdmins	Manage Tecres staff
`PATCH /alerts/:id/resolve`	canResolveAlert	

DEPRECATED

`POST /api/organizations` (public self-service org creation) still exists but contradicts the product model and is scheduled for removal (Security Remediation Register, item F-02). Do not build on it.

ENGINEERING

10 Email, files and background jobs

Email

`src/lib/email.ts` sends through the **Brevo HTTPS API** when `BREVO_API_KEY` is set (cloud hosts block SMTP ports), otherwise through SMTP when configured, otherwise it logs a warning and sends nothing. Templates are plain HTML in `src/emails/templates/` with `{{var}}` placeholders, wrapped by `layout.html` and carrying the organization's logo and color where relevant.

EMAIL	TRIGGER	LINK LIFETIME
Password reset	Reset request, or a platform owner's forced reset	60 minutes, single use
Organization invite	`POST /people/invite` and resend	Until accepted, revoked or expired
Platform admin invite	`POST /platform/admins/invite`	7 days

Files

`src/lib/storage.ts` stores images on **Cloudinary** when configured, other files on **S3** when configured, and falls back to PostgreSQL `bytea`. Every file row records its `org_id`, and `GET /files/:id` re-checks it on download.

Background jobs

JOB	INTERVAL	WHAT IT DOES
`alertDetection.cron.ts`	15 minutes	Raises platform alerts: asset quota at 90% (warning) or 100% (critical), expired plan, no activity for 30 days, failed-login spikes
`tokenCleanup.cron.ts`	60 minutes	Deletes expired refresh, reset and invite tokens, and refresh tokens revoked over 7 days ago

Jobs run with `setInterval` inside the API process. With more than one instance, each would run them; move them to a single scheduler before scaling out.

ENGINEERING

11 Local setup and configuration

Backend

```
cd asset-management-backend
pnpm install
cp .env.example .env           # fill in DATABASE_URL and the four 32+ character secrets
pnpm db:migrate
pnpm db:seed                   # optional: demo org "acme" + one super admin
pnpm db:seed-platform-admin    # optional: a platform owner for the console
pnpm dev                       # http://localhost:4000, health check at /health
```

Frontend

```
cd asset-management-frontend
npm install
echo VITE_API_BASE_URL=http://localhost:4000/api > .env   # no trailing slash needed
npm run dev                    # http://localhost:5173
```

Backend environment variables

Validated by Zod in `src/config/env.ts`; the process refuses to start if a required value is missing or a secret is under 32 characters.

VARIABLE	PURPOSE
`DATABASE_URL`	PostgreSQL connection string (required)
`JWT_ACCESS_SECRET`	Signs tenant access and pre-auth tokens
`JWT_REFRESH_SECRET`	Required at start-up but currently unused (refresh tokens are opaque random values, not JWTs)
`PLATFORM_ADMIN_JWT_SECRET`	Signs platform tokens; must differ from the tenant secret
`TOTP_ENCRYPTION_KEY`	Encrypts TOTP secrets at rest. **Changing it makes every enrolled authenticator unusable.**
`CORS_ALLOWED_ORIGINS`	Comma-separated frontend origins; the first is also used to build emailed links
`COOKIE_DOMAIN`	Domain for the refresh cookie
`TRUST_PROXY`	Number of reverse proxies in front (defaults to 1 in production). Setting it higher than reality lets clients spoof their IP past rate limits.
`BREVO_API_KEY`, `SMTP_*`	Email transport (optional in development)
`CLOUDINARY_*`, `AWS_*`	File storage (optional; falls back to the database)
`NODE_ENV`, `PORT`	Production turns on `Secure; SameSite=None` cookies and proxy trust

Production notes

- `pnpm build` compiles TypeScript and copies email templates and migrations into `dist/`; `pnpm start` applies migrations, then starts the server.
- The frontend is a static build (`npm run build` → `dist/`) served from `app.tecrestrackit.com`; it needs `VITE_API_BASE_URL` at build time.
- Because frontend and API are on different sites in production, the refresh cookie is `SameSite=None; Secure`, and the API's `CORS_ALLOWED_ORIGINS` must list the exact frontend origin.

ENGINEERING

12 Testing and quality gates

REPOSITORY	GATE	COMMAND
Backend	Type-check	`pnpm typecheck`
Backend	Integration + security suites (needs a separate test database, see README)	`pnpm test`
Frontend	Type-check (unused imports fail the build)	`npx tsc -b`
Frontend	Production build	`npm run build`
Frontend	Smoke test of pages that render real data	Playwright script against `npm run dev`, checking the console for errors

The backend suites run against the real Express app and a real PostgreSQL database, not mocks:

- `auth.integration.test.ts`: login, TOTP, refresh rotation and reuse detection, cross-org isolation, valuation hiding, global email uniqueness.
- `tokenPurpose.security.test.ts`: pre-auth tokens can never act as sessions.
- `test/security/pentest.security.test.ts`: 32 attack scenarios; open hardening items are recorded with `it.fails` (see the Security Remediation Register).

> **WHEN YOU MUST RUN THE BACKEND TESTS**
>
> Any change to authentication, an org-scoped query, token handling or the audit state machine. These are exactly the bug classes the backend was rebuilt to prevent.

Playwright is not a project dependency. Install it with `npm install --no-save playwright` for one-off checks, and confirm `git diff package.json package-lock.json` is empty before committing.

ENGINEERING

13 Conventions for contributors

Backend

- Each module is `<name>.router.ts` (HTTP wiring only) + `<name>.service.ts` (logic and database) + `<name>.schema.ts` (Zod).
- Every request body, query and params goes through `validate()`; never read raw `req.body`.
- Never accept `orgId` from a client. Use `scopedDb(orgId)` where it fits, and `createCrudRouter()` for plain catalog resources.
- Platform routes use only `authenticatePlatformAdmin` / `authorizePlatformAdmin`; tenant routes only `authenticate` / `authorize`. No route accepts both.
- New money fields go in `VALUATION_FIELDS` and through `stripValuationFields()`.
- Throw the typed errors from `src/lib/errors.ts` (`NotFoundError`, `ForbiddenError`, `ConflictError`, …) rather than writing responses by hand.

Frontend

- One folder per domain in `src/features`, with an `api.ts` of TanStack Query hooks; invalidate every affected key after a mutation.
- Keep types in `src/types` in step with the backend schema by hand.
- Route per-organization colors only through `--org-primary` / `--org-accent` and `applyOrgTheme`; support both light and dark palettes.
- Package managers: npm for the frontend, pnpm for the backend.

Before calling a change done

1. Backend: `pnpm typecheck` clean; `pnpm test` if you touched auth, scoping, tokens or audits.
2. Frontend: `npx tsc -b` clean and `npm run build` succeeds.
3. If a page renders real data, smoke-test it in a browser and check the console.
4. Update `BUILD_PLAN.md`'s status table if you started or finished a phase.